Network Attack Path Mapper: Team Brief

A short overview of the project for team members. For full details see PROJECT.md; for likely review questions see REVIEW_NOTES.md.

What the project does

If an intruder takes over one computer in a network, how far can they spread? This tool answers that question.

It takes three inputs:

- **hosts** (computers and their IP addresses);
- **open services** on each host (for example SSH on port 22, a web server on 80/443, a database on 5432);
- **firewall rules** between subnets.

From these it builds a **graph**: each computer is a node, and an arrow from A to B means "A can connect to a service on B". It then reports:

- which hosts can be reached, and in how many steps (hops);
- every route to the critical servers;
- **choke points**: hosts that every route must pass through, which are the best places to defend;
- what changes when the firewall policy changes.

The tool shows **reachability only**. It does not test whether a host can actually be broken into.

Key networking ideas

- **Different subnets:** traffic goes through the router, where the firewall checks its rules in order and the first match decides.
- **Same subnet:** traffic goes directly through the switch and **never reaches the firewall**, so no firewall rule can block it.
- **Allowed connections chain together.** Each rule can look harmless on its own, but together they can form a route to a critical server. That's what the tool is built to find.

Results so far

Area	Result
Sample network (7 hosts, 3 subnets)	Stricter firewall: connections **34 → 12**, routes to the database **50+ → 2**, database **1 → 2 hops** (only through the web server). The file server stays reachable because it shares a subnet with the database.
Real lab (4 virtual machines, real iptables firewall)	The tool's predictions matched **72 of 72** measured connections (95% confidence interval: 95.0–100%).
Large-scale tests (up to 1,000 hosts, 360 runs)	A normal network scan **missed 100%** of multi-hop routes; zone-level rule review gave **49% false alarms**; only this tool caught a misconfigured rule. 1,000 hosts are analysed in **under 15 seconds**.
Code quality	**41 automated tests**, all passing; one command (`.\reproduce.ps1`) regenerates every result.

Status: TRL 3. The idea is proven on paper and in a real lab.

What is new (novelty)

1. Finds **multi-hop** exposure, not only direct connections.
2. Models **traffic inside subnets**, which firewalls can't filter and most rule checks ignore.
3. **Validated on a real network:** the same policy file drives both the tool and the lab firewall.
4. **Measures how common methods fail** (100% missed, 49% false alarms).
5. Finds **choke points** to show where to defend first.

Project layout

Folder / file	Contents
`analyzer/`	Graph building, route search, choke points
`collector/`	Reads Nmap scan results
`visualization/`	Interactive network pictures (HTML)
`data/`	Sample network, lab network, firewall policies
`lab/`	Lab setup guide and scripts
`evidence/`	Lab measurements and screenshots
`experiments/`	Large-scale tests, results and charts
`tests/`	Automated tests
`main.py`	Command-line entry point

How to run it (Windows PowerShell)

From the project folder:

```
.\.venv\Scripts\python main.py compare --start 10.10.10.5 --after data/firewall_segmented.json
```

Then open `output\before.html` and `output\after.html` in a browser.

Reading the pictures:

- red star = starting host;
- orange diamonds = critical servers;
- dashed line = crosses the firewall;
- solid line = inside one subnet;
- thick red line = part of a route to a critical server.

Next steps

- **Hardening recommender:** suggest the smallest firewall change that blocks every route to the critical servers.
- **Larger validation:** test on a bigger, more realistic network (more hosts, UDP, host firewalls).